

QUESTION NO:01

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Vehicle {
```

```
private:
```

```
    string typeOfCar, make, model, color;
```

```
    int year;
```

```
    double milesDriven;
```

```
public:
```

```
    Vehicle(string t, string mk, string md, string c, int y, double m)
```

```
        : typeOfCar(t), make(mk), model(md), color(c), year(y), milesDriven(m) {}
```

```
    void displayVehicle() const {
```

```
        cout << "Type: " << typeOfCar << endl;
```

```
        cout << "Make: " << make << endl;
```

```
        cout << "Model: " << model << endl;
```

```
        cout << "Color: " << color << endl;
```

```
        cout << "Year: " << year << endl;
```

```
        cout << "Miles Driven: " << milesDriven << endl;
```

```
    }
```

```
};
```

```
class GasVehicle : virtual public Vehicle {
```

```
private:
```

```
    double fuelTankSize;
```

```
public:
```

```
    GasVehicle(string t, string mk, string md, string c, int y, double m, double fts)
```

```
        : Vehicle(t, mk, md, c, y, m), fuelTankSize(fts) {}
```

```
    void displayGas() const {
```

```
        cout << "Fuel Tank Size: " << fuelTankSize << endl;
```

```
    }
```

```
};
```

```
class ElectricVehicle : virtual public Vehicle {
```

```
private:
```

```
    double energyStorage;
```

```
public:
```

```
    ElectricVehicle(string t, string mk, string md, string c, int y, double m, double  
es)
```

```
        : Vehicle(t, mk, md, c, y, m), energyStorage(es) {}
```

```
void displayElectric() const {  
    cout << "Energy Storage: " << energyStorage << endl;  
}  
};
```

```
class HighPerformance : public GasVehicle {  
private:  
    int horsepower, topSpeed;  
  
public:  
    HighPerformance(string t, string mk, string md, string c, int y, double m,  
double fts, int hp, int ts)  
        : Vehicle(t, mk, md, c, y, m), GasVehicle(t, mk, md, c, y, m, fts),  
horsePower(hp), topSpeed(ts) {}
```

```
void displayHighPerformance() const {  
    cout << "Horse Power: " << horsepower << endl;  
    cout << "Top Speed: " << topSpeed << endl;  
}  
};
```

```
class SportsCar : public HighPerformance {  
public:
```

```
string gearbox, driveSystem;
```

```
SportsCar(string t, string mk, string md, string c, int y, double m, double fts,  
int hp, int ts, string gb, string ds)
```

```
: Vehicle(t, mk, md, c, y, m), HighPerformance(t, mk, md, c, y, m, fts, hp,  
ts), gearbox(gb), driveSystem(ds) {}
```

```
void displaySportsCar() const {
```

```
    displayHighPerformance();
```

```
    cout << "Gearbox: " << gearbox << endl;
```

```
    cout << "Drive System: " << driveSystem << endl;
```

```
}
```

```
};
```

```
class HeavyVehicle : public GasVehicle, public ElectricVehicle {
```

```
private:
```

```
    double maximumWeight, length;
```

```
    int numberOfWheels;
```

```
public:
```

```
    HeavyVehicle(string t, string mk, string md, string c, int y, double m, double  
fts, double es, double mw, int nw, double len)
```

```
    : Vehicle(t, mk, md, c, y, m), GasVehicle(t, mk, md, c, y, m, fts),  
ElectricVehicle(t, mk, md, c, y, m, es),
```

```
        maximumWeight(mw), numberOfWheels(nw), length(len) {}
```

```
void displayHeavy() const {  
    cout << "Maximum Weight: " << maximumWeight << endl;  
    cout << "Number of Wheels: " << numberOfWheels << endl;  
    cout << "Length: " << length << endl;  
}  
};
```

```
class ConstructionTruck : public HeavyVehicle {
```

```
public:
```

```
    string cargo;
```

```
    ConstructionTruck(string t, string mk, string md, string c, int y, double m,  
double fts, double es, double mw, int nw, double len, string cg)
```

```
        : Vehicle(t, mk, md, c, y, m), HeavyVehicle(t, mk, md, c, y, m, fts, es, mw,  
nw, len), cargo(cg) {}
```

```
void displayConstructionTruck() const {  
    displayHeavy();  
    cout << "Cargo: " << cargo << endl;  
}  
};
```

```
class Bus : public HeavyVehicle {  
private:  
    int numberOfSeats;  
  
public:  
    Bus(string t, string mk, string md, string c, int y, double m,  
        double fts, double es, double mw, int nw, double len, int ns)  
        : Vehicle(t, mk, md, c, y, m), HeavyVehicle(t, mk, md, c, y, m, fts, es, mw,  
nw, len), numberOfSeats(ns) {}  
  
    void display() const {  
        displayVehicle();  
        displayGas();  
        displayElectric();  
        displayHeavy();  
        cout << "Number of Seats: " << numberOfSeats << endl;  
    }  
};  
  
int main() {  
    Bus myBus("Passenger Bus", "Volvo", "9700", "White",  
        2023, 12500.5,  
        150.0, 300.0,
```

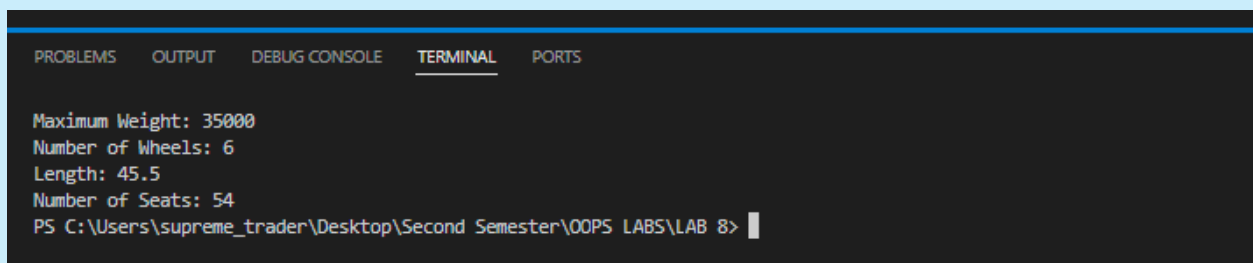
```
35000.0, 6, 45.5,  
54);
```

```
myBus.display();
```

```
return 0;
```

```
}
```

OUTPUT:

A screenshot of a C++ IDE's terminal window. The terminal has a dark background with a blue header bar containing tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected), and 'PORTS'. The output text in the terminal is: 'Maximum Weight: 35000', 'Number of Wheels: 6', 'Length: 45.5', and 'Number of Seats: 54'. Below the output, the command prompt shows the file path 'PS C:\Users\supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 8>' followed by a cursor.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
Maximum Weight: 35000  
Number of Wheels: 6  
Length: 45.5  
Number of Seats: 54  
PS C:\Users\supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 8> |
```

QUESTION NO:02

```
#include <iostream>
```

```
using namespace std;
```

```
class Character {
```

```
protected:
```

```
    string name;
```

```
    int level;
```

```
    int health;
```

public:

```
Character(string n, int l, int h) {
```

```
    name = n;
```

```
    level = l;
```

```
    health = h;
```

```
}
```

```
void display() {
```

```
    cout << "Name: " << name << endl;
```

```
    cout << "Level: " << level << endl;
```

```
    cout << "Health: " << health << endl;
```

```
}
```

```
};
```

```
class Warrior : virtual public Character {
```

protected:

```
    int strength;
```

```
    int meleeProficiency;
```

public:

```
Warrior(string n, int l, int h, int s, int m)
```

```
    : Character(n, l, h) {
```

```
        strength = s;
```

```
        meleeProficiency = m;
```

```
}
```



```
void slash() {  
    cout << "Slash attack!" << endl;  
}  
  
void display() {  
    Character::display();  
    cout << "Strength: " << strength << endl;  
    cout << "Melee Proficiency: " << meleeProficiency << endl;  
}  
};  
  
class Mage : virtual public Character {  
protected:  
    int intelligence;  
    int spellProficiency;  
  
public:  
    Mage(string n, int l, int h, int i, int sp)  
        : Character(n, l, h) {  
        intelligence = i;  
        spellProficiency = sp;  
    }  
  
    void fireball() {  
        cout << "Fireball cast!" << endl;  
    }  
};
```

```
}
```

```
void display() {
```

```
    Character::display();
```

```
    cout << "Intelligence: " << intelligence << endl;
```

```
    cout << "Spell Proficiency: " << spellProficiency << endl;
```

```
}
```

```
};
```

```
class Archer : public Character {
```

```
protected:
```

```
    int dexterity;
```

```
    int rangedProficiency;
```

```
public:
```

```
    Archer(string n, int l, int h, int d, int r)
```

```
        : Character(n, l, h) {
```

```
            dexterity = d;
```

```
            rangedProficiency = r;
```

```
}
```

```
void rapidShot() {
```

```
    cout << "Rapid shot!" << endl;
```

```
}
```

```
void display() {
```

```
    Character::display();  
    cout << "Dexterity: " << dexterity << endl;  
    cout << "Ranged Proficiency: " << rangedProficiency << endl;  
}  
};
```

```
class NPC : public Character {  
    string behavior;  
  
public:  
    NPC(string n, int l, int h, string b)  
        : Character(n, l, h) {  
        behavior = b;  
    }  
}
```

```
void display() {  
    Character::display();  
    cout << "Behavior: " << behavior << endl;  
}  
};
```

```
class Mighty : public Warrior, public Mage {  
public:  
    Mighty(string n, int l, int h, int s, int m, int i, int sp)  
        : Character(n, l, h),  
        Warrior(n, l, h, s, m),
```

```
Mage(n, l, h, i, sp) {}
```

```
void display() {
```

```
    Character::display();
```

```
    cout << "Strength: " << strength << endl;
```

```
    cout << "Melee Proficiency: " << meleeProficiency << endl;
```

```
    cout << "Intelligence: " << intelligence << endl;
```

```
    cout << "Spell Proficiency: " << spellProficiency << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
    Mighty m("Hero", 10, 100, 80, 70, 90, 85);
```

```
    m.display();
```

```
    m.slash();
```

```
    m.fireball();
```

```
    return 0;
```

```
}
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Name: Hero
Level: 10
Health: 100
Strength: 80
Melee Proficiency: 70
Intelligence: 90
Spell Proficiency: 85
Slash attack!
Fireball cast!
PS C:\Users\supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 8> 
```

QUESTION NO:03

```
#include <iostream>

using namespace std;
```

```
class Account {
```

```
protected:
```

```
    double balance;
```

```
public:
```

```
    Account() {
```

```
        cout << "Enter initial balance: ";
```

```
        cin >> balance;
```

```
    }
```

```
    Account(double b) {
```

```
        balance = b;
```

```
    }
```

```
    virtual void deposit(double amount) {
```

```
        balance += amount;
    }

    virtual void withdraw(double amount) {
        if (amount <= balance)
            balance -= amount;
        else
            cout << "Insufficient balance\n";
    }

    void checkBalance() const {
        cout << "Current Balance: Rs. " << balance << endl;
    }
};

class InterestAccount : virtual public Account {
protected:
    double interest;

public:
    InterestAccount() : Account() {
        char choice;
```

```
    cout << "Do you want to enter a custom interest rate? (y/n) [Default is 0.30]: ";
```

```
    cin >> choice;
```

```
    if (choice == 'y' || choice == 'Y') {
```

```
        cout << "Enter interest rate (e.g., 0.30 for 30%): ";
```

```
        cin >> interest;
```

```
    } else {
```

```
        interest = 0.30;
```

```
    }
```

```
}
```

```
InterestAccount(double b, double i = 0.30) : Account(b) {
```

```
    interest = i;
```

```
}
```

```
void deposit(double amount) override {
```

```
    balance += amount + (amount * interest);
```

```
    cout << "Deposited Rs. " << amount << " with interest applied.\n";
```

```
}
```

```
};
```

```
class ChargingAccount : virtual public Account {
```

```
protected:
```

```
double fee;
```

```
public:
```

```
    ChargingAccount() : Account() {  
        char choice;  
        cout << "Do you want to enter a custom withdrawal fee? (y/n) [Default is  
Rs. 25]: ";  
        cin >> choice;  
        if (choice == 'y' || choice == 'Y') {  
            cout << "Enter withdrawal fee: ";  
            cin >> fee;  
        } else {  
            fee = 25.0;  
        }  
    }  
}
```

```
    ChargingAccount(double b, double f = 25.0) : Account(b) {  
        fee = f;  
    }  
}
```

```
void withdraw(double amount) override {  
    double total = amount + fee;
```



```
    if (total <= balance) {  
        balance -= total;  
        cout << "Withdrew Rs. " << amount << " (Fee applied: Rs. " << fee <<  
        ").\n";  
    } else {  
        cout << "Insufficient balance to cover amount + fee.\n";  
    }  
}  
};
```

```
class ACI : public InterestAccount, public ChargingAccount {  
public:
```

```
    ACI() : Account(), InterestAccount(), ChargingAccount() {}
```

```
    ACI(double b)
```

```
        : Account(b), InterestAccount(b), ChargingAccount(b) {}
```

```
    void transfer(double amount, Account &acc) {
```

```
        if (amount <= balance) {
```

```
            balance -= amount;
```

```
            acc.deposit(amount);
```

```
        cout << "Successfully transferred Rs. " << amount << " to Account.\n";
    } else {
        cout << "Transfer failed: Insufficient balance\n";
    }
}
```

```
void transfer(double amount, InterestAccount &acc) {
    if (amount <= balance) {
        balance -= amount;
        acc.deposit(amount);
        cout << "Successfully transferred Rs. " << amount << " to Interest
Account.\n";
    } else {
        cout << "Transfer failed: Insufficient balance\n";
    }
}
```

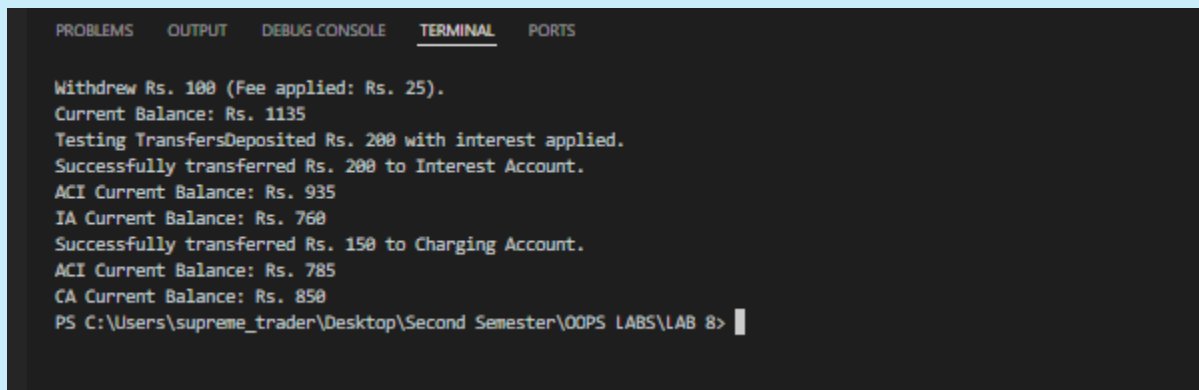
```
void transfer(double amount, ChargingAccount &acc) {
    if (amount <= balance) {
        balance -= amount;
        acc.deposit(amount);
        cout << "Successfully transferred Rs. " << amount << " to Charging
Account.\n";
    } else {
```

```
        cout << "Transfer failed: Insufficient balance\n";  
    }  
}  
};
```

```
int main() {  
    cout << "--- Initializing Accounts ---\n";  
    ACI a1(1000);  
    InterestAccount ia(500);  
    ChargingAccount ca(700);  
  
    cout << "\n--- Testing ACI Functionality ---\n";  
    a1.checkBalance();  
  
    a1.deposit(200);  
    a1.checkBalance();  
  
    a1.withdraw(100);  
    a1.checkBalance();  
  
    cout << "Testing Transfers";  
    a1.transfer(200, ia);
```

```
cout << "ACI "; a1.checkBalance();  
cout << "IA "; ia.checkBalance();  
  
a1.transfer(150, ca);  
cout << "ACI "; a1.checkBalance();  
cout << "CA "; ca.checkBalance();  
  
return 0;  
}
```

OUTPUT:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
Withdraw Rs. 100 (Fee applied: Rs. 25).  
Current Balance: Rs. 1135  
Testing TransfersDeposited Rs. 200 with interest applied.  
Successfully transferred Rs. 200 to Interest Account.  
ACI Current Balance: Rs. 935  
IA Current Balance: Rs. 760  
Successfully transferred Rs. 150 to Charging Account.  
ACI Current Balance: Rs. 785  
CA Current Balance: Rs. 850  
PS C:\Users\supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 8> |
```

QUESTION NO:04

```
#include <iostream>  
  
#include <string>  
  
using namespace std;  
  
class Media {  
protected:  
    string title;
```

```
bool isBorrowed;
```

```
public:
```

```
Media(string t) {
```

```
    title = t;
```

```
    isBorrowed = false;
```

```
}
```

```
void borrowMedia() {
```

```
    if (!isBorrowed) {
```

```
        isBorrowed = true;
```

```
        cout << title << " borrowed\n";
```

```
    } else {
```

```
        cout << title << " already borrowed\n";
```

```
    }
```

```
}
```

```
void returnMedia() {
```

```
    if (isBorrowed) {
```

```
        isBorrowed = false;
```

```
        cout << title << " returned\n";
```

```
    } else {
```

```
        cout << title << " was not borrowed\n";
```

```
    }
```

```
}
```

```
virtual void display() const {  
    cout << "Title: " << title << endl;  
    cout << "Status: " << (isBorrowed ? "Borrowed" : "Available") << endl;  
}  
};
```

```
class BookDetails {  
protected:  
    string author;  
  
public:  
    BookDetails(string a) {  
        author = a;  
    }  
};
```

```
class MagazineDetails {  
protected:  
    int issueNumber;  
  
public:  
    MagazineDetails(int i) {  
        issueNumber = i;  
    }  
};
```

```
class DVDDetails {  
protected:  
    string director;  
  
public:  
    DVDDetails(string d) {  
        director = d;  
    }  
};  
  
class Book : public Media, public BookDetails {  
public:  
    Book(string t, string a) : Media(t), BookDetails(a) {}  
  
    void display() const override {  
        Media::display();  
        cout << "Author: " << author << endl;  
    }  
};  
  
class Magazine : public Media, public MagazineDetails {  
public:  
    Magazine(string t, int i) : Media(t), MagazineDetails(i) {}  
  
    void display() const override {  
        Media::display();
```

```
        cout << "Issue Number: " << issueNumber << endl;
    }
};
```

```
class DVD : public Media, public DVDDetails {
public:
    DVD(string t, string d) : Media(t), DVDDetails(d) {}

    void display() const override {
        Media::display();
        cout << "Director: " << director << endl;
    }
};
```

```
int main() {
    Book b("1984", "George Orwell");
    Magazine m("Time", 542);
    DVD d("Matrix", "Wachowski");

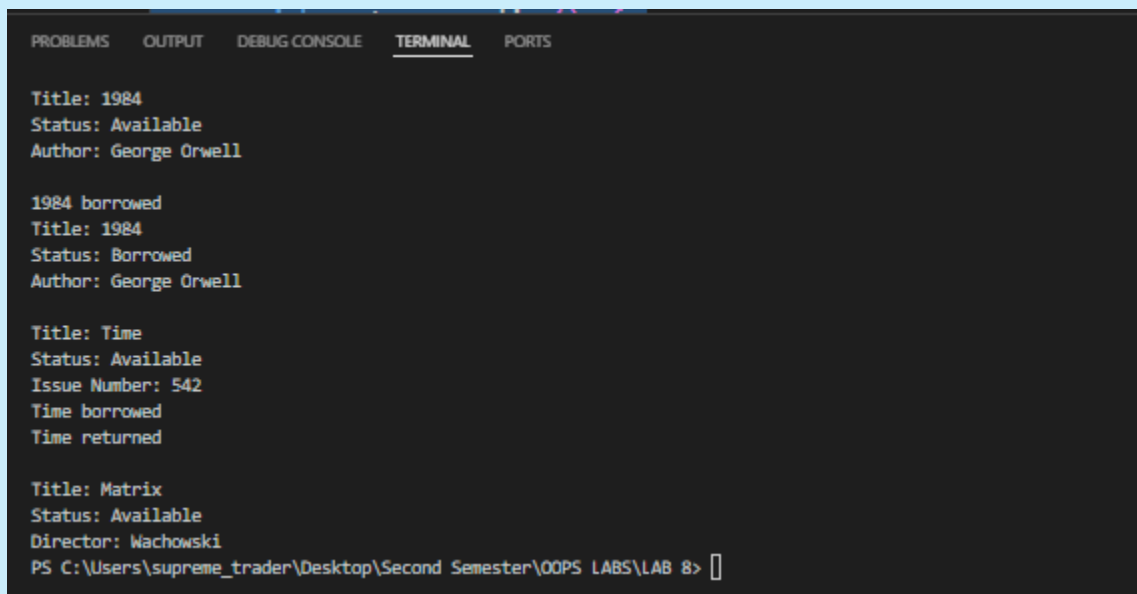
    b.display();
    cout << endl;

    b.borrowMedia();
    b.display();
    cout << endl;
```



```
m.display();  
m.borrowMedia();  
m.returnMedia();  
cout << endl;  
  
d.display();  
  
return 0;  
}
```

OUTPUT:



The screenshot shows a terminal window with the following output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
Title: 1984  
Status: Available  
Author: George Orwell  
  
1984 borrowed  
Title: 1984  
Status: Borrowed  
Author: George Orwell  
  
Title: Time  
Status: Available  
Issue Number: 542  
Time borrowed  
Time returned  
  
Title: Matrix  
Status: Available  
Director: Wachowski  
PS C:\Users\supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 8> |
```